Estructuras

En ciertos casos necesitamos crear nuestros propios tipos de datos, necesarios para desarrollar el programa que tenemos planeado y tal vez los tipos de datos nativos no son suficientes para nuestro desarrollo. Para eso contamos con las estructuras que nos permiten crear un tipo de dato propio el cual tiene atributos definidos por tipos de datos nativos

Una estructura en C, definida por la palabra clave ***struct***, es un tipo de dato compuesto que permite agrupar diferentes tipos de datos bajo un solo nombre. Cada componente dentro de la estructura se denomina miembro o campo, y puede ser de cualquier tipo de dato (incluyendo otros struct). Las estructuras son útiles cuando se necesita representar un objeto o entidad con múltiples atributos de diferentes tipos.

Sintaxis

```
struct nombre_estructura {
tipo_dato1 nombre_campo1;
tipo_dato2 nombre_campo2;
// Otros campos
};
```

- struct: Palabra clave que indica que estamos definiendo una estructura.
- nombre_estructura: El nombre que se le asigna a la estructura.
- tipo_dato1, tipo_dato2: Los tipos de datos de cada campo.
- nombre_campo1, nombre_campo2: Los nombres de los campos dentro de la estructura.

Una vez definida la estructura, se pueden declarar variables de ese tipo de la siguiente manera:

```
struct Persona persona1;
```

Ahora cada vez que tenemos que crear una variable representada por esta estructura necesitamos recurrir a struct, esto lo podemos evitar y simplificar utilizando la palabra clave ***typedef*** y definiendo el tipo de dato clave en la propia definición de la estructura, veamos el ejemplo:

```
typedef struct
{
    int codigo;
    char nombre[20];
}tagenda;

int main()
{
    tagenda contactos;
    return 0;
}
```

De esta manera queda definido el tipo de dato general tagenda que corresponde a una estructura del tipo definido.

Acceso a los campos o atributos

Una vez definido el tipo de dato base para utilizar nuestra estructura, necesitamos crear variables y acceder a los campos. Estas variables pueden ser punteros o variables estáticas, pero según sea una u otra, la manera de acceder a los campos o atributos varia. En el caso de una estructura creada como puntero podemos acceder a los campos con ***"->"***, pero si la variable estructura creada es estática podemos acceder mediante ***"."***, veamos los ejemplos:

```
typedef struct
{
    int codigo;
    char nombre[20];
}tagenda;

int main()
{
    tagenda contactos;
    tagenda *contactos2=(tagenda*)malloc(sizeof(tagenda));

    contactos.codigo=12;//con punto para estaticas
    contactos2->codigo=24;//con flecha para punteros
    return 0;
}
```

Algunas aplicaciones de estructuras

- Como argumentos de funciones:

```
void imprimir_contacto(tagenda * cl)
{
    printf("Codigo: %d\n",cl->codigo);
    printf("Nombre: %s\n",cl->nombre);
}
```

- Arreglos de estructuras

```
tagenda contactos[3] = {
    {30,"Juan"},
    {25,"Ana"},
    {40,"Pedro"}
};
```